



DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA,
ELETTRONICA E SISTEMISTICA

Robotics and automation engineering

Dynamical Systems theory course

**Modeling and control of human
postural stability : a state space
approach via inverted pendulum
dynamics**

Professore:

Prof.

Alessandro Casavola

Studentessa:

Francesca Vaccaro

Mat. 276582

ANNO ACCADEMICO 12/34

Indice

Indice	1
1 Introduction	3
1.1 Presentation of the topic and goal of the project	3
1.2 Tools used	4
2 Mathematical model	5
2.1 Description of mathematical model	5
2.1.1 Parameter values	7
2.1.2 Simulink and Matlab implementation	8
Second differential equation	8
First differential equation	10
3 Equilibrium point and linearization	12
3.0.1 Meaning of linearization	13
3.0.2 Equilibrium points	14
3.0.3 Linearization of the system	15
4 System Analysis	17
4.1 Eigenvalues and Eigenvectors	18
4.2 System evolution	19
Free Response	19
Forced Response	20
5 Synthesis	22
5.1 Structural properties	22
5.1.1 Reachability	22

Matlab implementation	24
5.1.2 Observability	24
Matlab implementation	25
5.2 Static State feedback	26
5.2.1 Matlab and simulink	27
5.3 Asymptotic observer(Luenberger)	32
6 Conclusion	38
Bibliografia	40

Capitolo 1

Introduction

1.1 Presentation of the topic and goal of the project

The *human body* can be interpreted as a *complex dynamical system*, consisting of multiple interacting subsystems that ensure the organism's proper functioning. Among the various phenomena of interest, the *maintenance of upright posture* represents a particularly relevant problem from the perspective of dynamical systems and control theory, as the vertical configuration of the human body corresponds to an inherently *unstable equilibrium condition*. During quiet standing, the body is continuously subject to internal and external perturbations that would lead to a loss of balance if not appropriately compensated. Postural stability is made possible by the combined action of the nervous and muscular systems, which implement a feedback control mechanism capable of generating corrective torques at the joints, particularly at the ankle.

To make the problem analytically tractable, the human body is modeled as an inverted pendulum hinged at the ankle. This approximation is widely utilized in biomechanical literature and allows for capturing the primary dynamical characteristics of postural control during standing. The model parameters adopted in this work are consistent with anthropometric values reported in previous experimental studies, specifically following the standards established by Winter [1].

The first part of the project is dedicated to the mathematical modeling of the system, with the objective of translating biomechanical behavior into a set of non-linear differential equations. In this phase, the fundamental properties of the dynamical system are analyzed, supporting the theoretical discussion through numerical simulations performed in MATLAB and Simulink.

Subsequently, the equilibrium points of the system are determined, and linearization is performed around the equilibrium of interest. The linearized state-space representation allows for the application of classical tools for linear dynamical systems analysis, such as the study of stability, structural properties (reachability and observability), and time-domain responses.

The final part of the project is dedicated to the design of a feedback controller aimed at stabilizing the upright posture and rejecting perturbations. Finally, the behavior of the controlled system is analyzed through simulations to evaluate the effectiveness of the adopted control strategy, such as Full State Feedback (FSF).

1.2 Tools used

As mentioned previously, in order to carry out the various calculations, view the work and observe the simulations, it was used a very powerful software : **MATLAB**, with its instrument, like for example *simulink*.

Capitolo 2

Mathematical model

2.1 Description of mathematical model

As I previously said, mathematical model has been defined on the basis of the simplification of the human body as an inverted pendulum; the image shows all the associations between the two models.

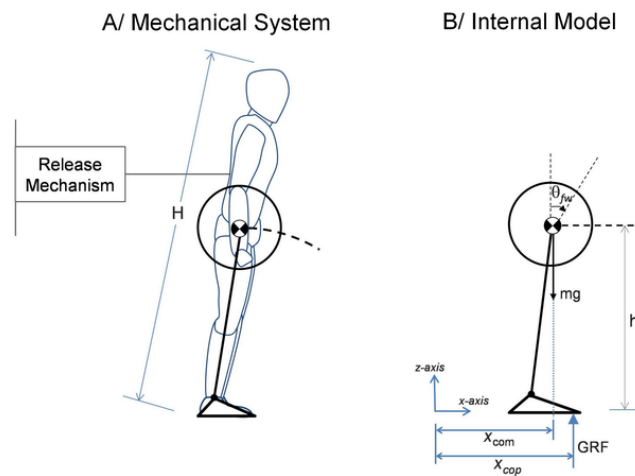


Figura 2.1: Association body-inverted pendulum

But, in order to derive in a correct way the mathematical equations it's important to pay attention to the differences between COM and COP that are respectively the Center of Mass and the Center of Pressure.

The first one is the net position of the center of mass in the three-dimensional space and it's computed as the weighted sum of COM for each segment written in the table :

Segment	Definition	Segment	Center of Mass/ Segment Length	
		Weight/Total Body Weight	Proximal	Distal
Hand	Wrist axis/knuckle II middle finger	0.006 M	0.506	0.494 P
Forearm	Elbow axis/ulnar styloid	0.016 M	0.430	0.570 P
Upper arm	Glenohumeral axis/elbow axis	0.028 M	0.436	0.564 P
Forearm and hand	Elbow axis/ulnar styloid	0.022 M	0.682	0.318 P
Total arm	Glenohumeral joint/ulnar styloid	0.050 M	0.530	0.470 P
Foot	Lateral malleolus/head metatarsal II	0.0145 M	0.50	0.50 P
Leg	Femoral condyles/medial malleolus	0.0465 M	0.433	0.567 P
Thigh	Greater trochanter/femoral condyles	0.100 M	0.433	0.567 P
Foot and leg	Femoral condyles/medial malleolus	0.061 M	0.606	0.394 P
Total leg	Greater trochanter/medial malleolus	0.161 M	0.447	0.553 P
Head and neck	C7-T1 and 1st rib/ear canal	0.081 M	1.000	— PC
Shoulder mass	Sternoclavicular joint/glenohumeral axis	—	0.712	0.288
Thorax	C7-T1/T12-L1 and diaphragm*	0.216 PC	0.82	0.18
Abdomen	T12-L1/L4-L5*	0.139 LC	0.44	0.56
Pelvis	L4-L5/greater trochanter*	0.142 LC	0.105	0.895
Thorax and abdomen	C7-T1/L4-L5*	0.355 LC	0.63	0.37
Abdomen and pelvis	T12-L1/greater trochanter*	0.281 PC	0.27	0.73
Trunk	Greater trochanter/glenohumeral joint*	0.497 M	0.50	0.50
Trunk head neck	Greater trochanter/glenohumeral joint*	0.578 MC	0.66	0.34 P
Head, arms, and trunk (HAT)	Greater trochanter/glenohumeral joint*	0.678 MC	0.626	0.374 PC
HAT	Greater trochanter/mid rib	0.678	1.142	—

Figure 2.2: Table of center of mass

The base of support can vary from one foot to a four point support, and it's essential that the COM remain within that base of support or move safely between the two feet if it lies temporarily outside the base of support (as it does in running and during the single-support phase of walking).

The inverted pendulum model allows us to relate COP and COM. The position of the COP relative at the COM decide direction of the angular acceleration of the pendulum, so, assuming that the body swings around the ankles we reach this relation:

$$COP_x - COM_x = -\frac{I_s C\ddot{O}M_x}{Wh} \quad (2.1)$$

Where :

1. I_s = moment of inertia of the body about ankles.
2. COM_x = forward acceleration of COM.
3. W = body weight.
4. h = height of COM above ankles.

From this, considering :

1. θ = angle

2. $\dot{\theta} = \omega$ angular velocity
3. $\dot{\omega} =$ angular acceleration
4. $\tau_a =$ muscle activation torque

We can obtain the first *differential equation* referring to the **body dynamics** :

$$\begin{cases} \dot{\theta} = \omega \\ I_s \dot{\omega} = m \cdot g \cdot h \cdot \sin(\theta) - \beta \omega - \tau_a \end{cases} \quad (2.2)$$

Now, in addition to the dynamics of the body, I decided to also consider the neuro-muscular process as an actuator of the system. [3]

In order to do this, I introduced the input that arrives from the neural cells (u), and a time constant referring to how fast the input is followed, and call this parameter T_{mus} ; if we combine everything we obtain this system :

$$\begin{cases} \dot{\theta} = \omega \\ I_s \dot{\omega} = m \cdot g \cdot h \cdot \sin(\theta) - \beta \omega - \tau_a \\ \dot{\tau}_a = -\frac{\tau_a}{T_{mus}} + \frac{u}{T_{mus}} \end{cases} \quad (2.3)$$

2.1.1 Parameter values

Now, it's time to assign some value at the parameters already defined :

Tabella 2.1: Parameters of the anthropometric model (Subject: 52 kg, 1.60 m)

Parameter	Description	Value	Unit
m	Total body mass	52	kg
H	Total height	1.60	m
g	Gravity acceleration	9.81	m/s ²
h	Height of COM ($H \cdot 0.567$)	0.907	m
I_s	Moment of inertia at the ankle	21.68	kg·m ²
β	Passive damping coefficient	3.5	Nms/rad
T_{mus}	Muscle time constant	0.1	s

2.1.2 Simulink and Matlab implementation

At this point we can implement in MATLAB and in simulink.

At the first we define the parameters in a *.m* file :

```
1 %% PARAMETERS OF THE MODEL
2
3 %H = height of the person
4 H = 1.60;           %[m]
5 %mass
6 m = 52;             %[kg]
7 %gravity
8 g = 9.81;           %[N/kg]
9 %antropometric constant computed by Winter
10 K = 0.448;
11
12 %h = H * 0.567 is the height of COM
13 h = H * 0.567;
14 %moment of inertia of the ankle
15 Is = m * (K*H)^2;
16 %viscosity
17 beta = 3.5;
18 Tmus = 0.1;
```

Listing 2.1: MATLAB Script for State-Space Analysis

For the implementation in Simulink, I preferred to divide the two differential equations into two communicating subsystems, since the second differential equation had an input for the first one (τ_a) I implement in the first this and then the other one.

Second differential equation

In order to create a subsystem, in simulink, I defined in the first all the blocks that I need and then I've assembled all together, the only elements out are the input(signal that the system receives from the brain), and the output that goes in the other subsystem.

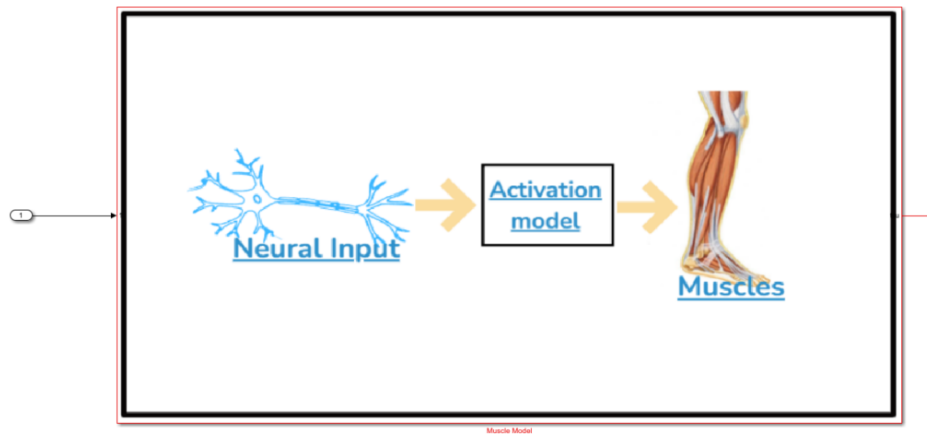


Figura 2.3: Muscle subsystem

If I double-click on this, I can show the various blocks that make up the differential equation :

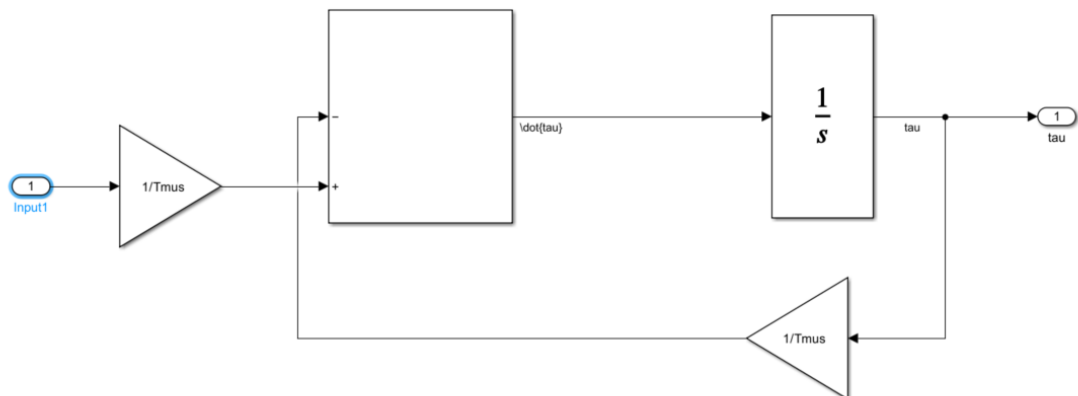


Figura 2.4: Muscle subsystem - showing blocks

As we can see, this represents in every detail the differential equation, in particular the main elements used in this case are :

1. Gain.
2. Integrator.
3. Sum.
4. Input.

5. Output.

The muscle activation subsystem implements the first-order dynamics of the torque τ_a . By rearranging the equation as $\dot{\tau}_a = \frac{1}{T_{mus}}(u - \tau_a)$, the model clearly separates the external neural command from the internal feedback of the current activation state

First differential equation

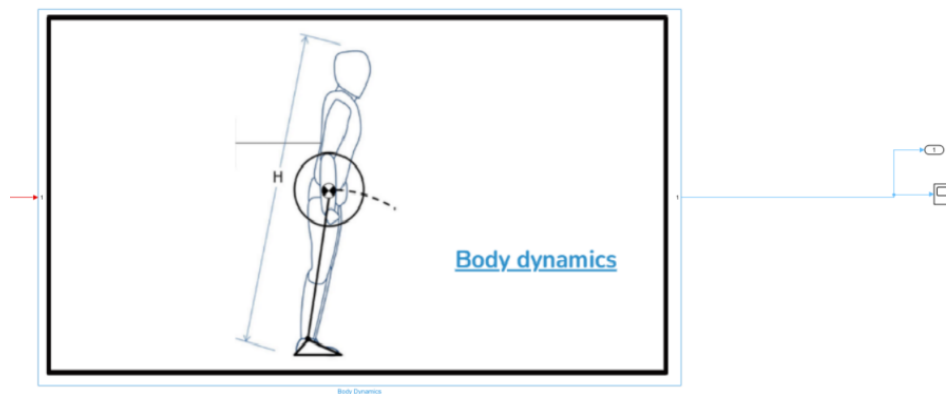


Figura 2.5: Body dynamics subsystem

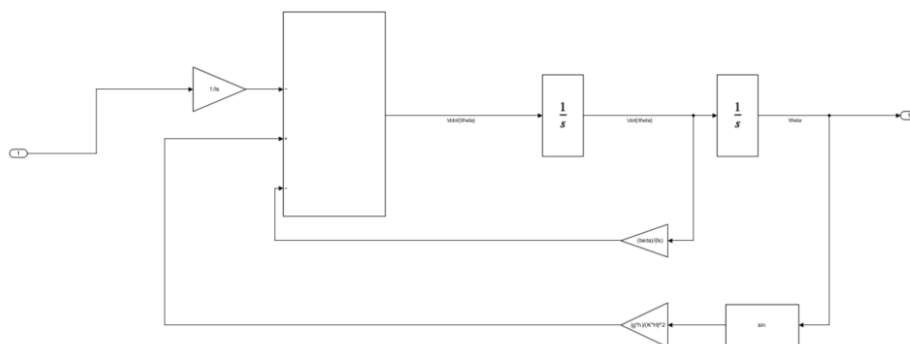


Figura 2.6: Body dynamics subsystem-blocks

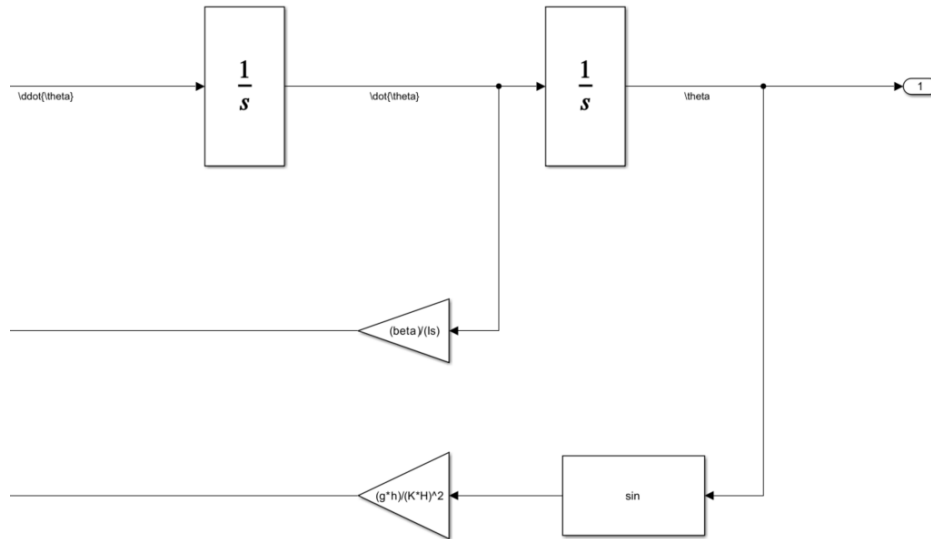


Figura 2.7: Body dynamics subsystem-blocks-zoom

The entire system, with the communication by the two compartment of body dynamics and muscle activation :

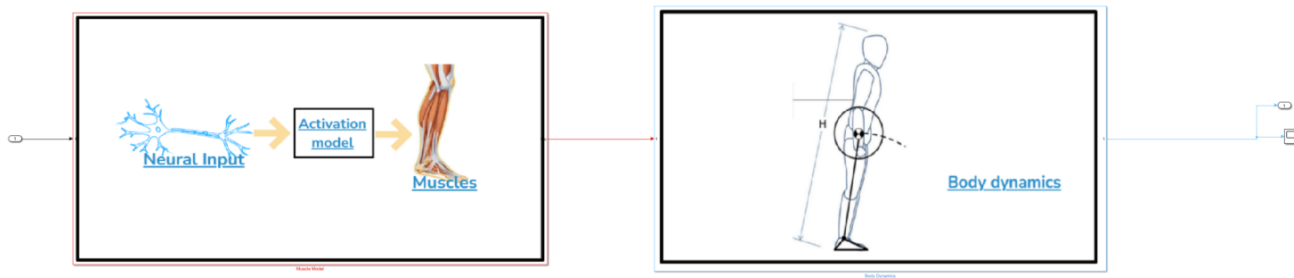


Figura 2.8: Body dynamics subsystem-blocks-zoom

Capitolo 3

Equilibrium point and linearization

To analyze the system and subsequently build a controller, it's important to accurately visualize how it's structured.

Let's start with the system of differential equations already proposed in the previous chapter:

$$\begin{cases} \dot{\theta} = \omega \\ I_s \dot{\omega} = m \cdot g \cdot h \cdot \sin(\theta) - \beta \omega - \tau_a \\ \dot{\tau}_a = -\frac{\tau_a}{T_{mus}} + \frac{u}{T_{mus}} \end{cases} \quad (3.1)$$

Now, let's define a dynamical system, like a black box that receives an **input** (and other signal like for example some disturbance), has a **state** and response with an **output**, those elements could be written under a vectorial representation and put together for a different representation of the system.

In this case, the **input** is very clear, because it is also specified and is $u(t)$, then, the **states** are three, and can be written in this way :

$$x_1 = \theta, \quad x_2 = \omega, \quad x_3 = \tau_a$$

The thing that we can build at this point is the **I/S/O Locally Representation** and in order to do this we need to refer to the derivative terms because we are talking about a *continuous time system*.

In particular we want to reach this type of configuration :

$$\begin{cases} \dot{x} = f(t, x(t), u(t)) \\ y = \eta(t, x(t), u(t)) \end{cases} \quad (3.2)$$

So our state vector began :

$$\dot{x} = \begin{pmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{pmatrix} \quad (3.3)$$

Where :

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = \frac{mgh \sin(x_1)}{I_s} - \frac{\beta x_2}{I_s} - \frac{x_3}{I_s} \\ \dot{x}_3 = -\frac{x_3}{T_{mus}} + \frac{u}{T_{mus}} \end{cases} \quad (3.4)$$

As we can see from this form, the system *is not linear*, so in order to do all the analysis and to bring the system in state space form we need to linearize around a point.

3.0.1 Meaning of linearization

Linearize means find a linear model that approximate the behaviour of linear model locally.

In order to understand in deep this concept, is important to introduce the meaning of *nominal solution* and *perturbated solution*.

The first one is the value of the function in the point of interest

$$\hat{x}_0, \hat{u}(\cdot) \rightarrow \hat{x}(t), \hat{y}(t)$$

The second one add some perturbation like for example an error value :

$$x_0, u(\cdot) \rightarrow x(t), y(t)$$

The error is the difference:

$$\tilde{x}(t) = x(t) - \hat{x}(t)$$

When we linearize a function, we do a **Taylor expansion**, because it's like we decompose the system in a sum of small perturbation while neglecting higher-order terms, provided that the perturbations $\tilde{x}$ and $\tilde{u}$ remain small.

$$\dot{\hat{x}}(t) + \dot{\tilde{x}}(t) = f(\hat{x}(t), \hat{u}(t)) + \left. \frac{\partial f(x, u)}{\partial x} \tilde{x}(t) \right|_{x=\hat{x}(t), u=\hat{u}(t)} + \left. \frac{\partial f(x, u)}{\partial u} \tilde{u}(t) \right|_{x=\hat{x}(t), u=\hat{u}(t)} + o(\tilde{x}(t), \tilde{u}(t)) \quad (3.5)$$

And the same is valid for the output.

3.0.2 Equilibrium points

The fact is that the linearization of the system if is done round a certain type of points, that are for example the equilibrium points; but what are the eq points and how we can compute that.

We have to take the nominal solution, and select input as constant $\hat{u}(t) = \hat{\bar{u}}$, now we have to impose the function computed in the x_{eq} , $\hat{\bar{u}}$ to 0 :

$$0 = f(x_{eq}, \hat{\bar{u}}) \Rightarrow \hat{x}(t) \equiv x_{eq} \quad (3.6)$$

Let's apply this to our system, the first application is the easiest one, because we impose $\dot{x}_1$ equal to 0, this depends only from x_{2eq} so this is 0, then we consider the third equation('cause the second one depends both from x_{1eq} and x_{3eq}) :

$$0 = -\frac{x_3}{T_{mus}} + \frac{u}{T_{mus}} \Rightarrow x_{3eq} = 0$$

If we solve also the second one we obtain two results $x_{1eq} = 0$, $x_{1eq} = \pi$.

So now it's possible to write the equilibrium points that we found :

$$X_{eq}^1 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}, \quad X_{eq}^2 = \begin{pmatrix} \pi \\ 0 \\ 0 \end{pmatrix} \quad (3.7)$$

This thing could be done also on matlab with the command *trim()* :

```
1 %% INITIAL CONDITIONS
2
3 % theta = 0 (vertical), omega = 0 , tau_a = 0 (muscle relaxed)
```

```

4 x0 = [0 0 0]';
5 u0 = 0;
6 y0 = x0;
7 idx = [1];
8
9 %% EQUILIBRIUM POINT
10
11 %punti di equilibrio
12 [xeq,ueq] = trim('MathematicalModel',x0,u0,y0,idx,[],[])

```

Listing 3.1: MATLAB Script for State-Space Analysis

Result is :

```

xeq =
    0
    0
    0

ueq =
    0

```

Figura 3.1: Muscle subsystem - showing blocks

That is exactly one of the point that we found.

3.0.3 Linearization of the system

Now it's time to linearize round the equilibrium point founded, in order to do this we're going to use the **jacobian matrix** that is the matrix composed by the partial derivation of the faction related to the state :

$$A = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \cdots & \cdots & \cdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix} \quad (3.8)$$

Same is for B, C and D.

We can do this directly in matlab with the function *linmod()* :

```

1 %linearization of the model
2 [A,B,C,D] = linmod('MathematicalModel',xeq,ueq)
3
4 %state space representation
5 sys = ss(A,B,C,D)

```

Listing 3.2: MATLAB Script for State-Space Analysis

Results :

```

1 A =
2      0      1.0000      0
3  17.3211  -0.1310  -0.0374
4      0      0  -10.0000
5
6 B =
7      0
8      0
9     10
10
11 C =
12      1      0      0
13
14 D =
15      0

```

Listing 3.3: MATLAB Script for State-Space Analysis

Capitolo 4

System Analysis

For the system analysis let's write in the state space form, and remember that this representation is possible only if we are treating a *linear* system :

Starting from the local I/S/U representation :

$$\begin{cases} \dot{x}(t) = f(t, x, u) \\ y = \eta(t, x, u) \end{cases} \quad (4.1)$$

Where : $u \in \mathbb{R}^m$, $x \in \mathbb{R}^n$, $y \in \mathbb{R}^p$ So if we draw the vector using for axes the canonical vectors, we can express the vectors in this way :

$$x = \sum_{i=1}^n x_i e_i, \quad u = \sum_{i=1}^m u_i e_i, \quad y = \sum_{i=1}^p y_i e_i \quad (4.2)$$

Now we can express A in this way :

$$A = \begin{bmatrix} f(t, e_1, 0) & f(t, e_2, 0) & \dots & f(t, e_n, 0) \end{bmatrix} \in \mathbb{R}^{n \times n} \quad (4.3)$$

In this case as it's possible to observe, the input u is not considering, and the same is done also for the computation of B,C,D, obviously, for B and D matrix the state is not considered, and for C also the input.

In the end we get this system :

$$\begin{cases} \dot{x} = Ax + Bu \\ y = Cx + Du \end{cases} \quad (4.4)$$

The matrices are the ones that we found in the previous chapter :

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 17.3211 & -0.1310 & -0.0374 \\ 0 & 0 & -10 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \\ 10 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}, \quad D = 0 \quad (4.5)$$

4.1 Eigenvalues and Eigenvectors

Given a square matrix, in my case the state matrix $A \in \mathbb{R}^{n \times n}$, a scalar number λ (real or complex) is said **eigenvalues** of A if exists a vector $v \in \mathbb{R}^n$ called **eigenvector** such that this relation is valid :

$$Av = v\lambda \quad (4.6)$$

To find values of λ the equation is written in this way :

$$(A - \lambda I)v \quad (4.7)$$

And then we have to solve the characteristic polynomial $\det(A - \lambda I)$, if we do this we obtain all the eigenvalues of A , and in order to check if our work is correct, is possible to use an operation in matlab :

```
1 eigA = eig(A)
```

Listing 4.1: MATLAB Script for State-Space Analysis

The result is : $\lambda_1 = 4.0969$, $\lambda_2 = -4.2279$, $\lambda_3 = -10$

As it's possible to observe there are two eigenvalues negative and one positive, for this reason system is *unstable*.

In order to understand well what is the meaning of this, it's important to introduce some concepts that are expressed in the ***Reduced Lyapunov's stability criterium***, it said that : given a system with this form : $\dot{x}(t) = Ax(t)$ and with A linearized round the equilibrium point $A = \left. \frac{\partial f}{\partial x} \right|_{x=x_{eq}}$, then, if :

1. matrix A has all eigenvalues stable, and so $Re(\lambda) < 0$ then we can say that x_{eq} is locally asymptotically stable for the non linear system.

2. matrix A has at least one eigenvalues with real part positive, so $Re(\lambda) > 0$ then the equilibrium point is unstable for the non linear system.
3. in all other situation we cannot say anything.

Based on this information, the case of this project is the second one, so system unstable, needs to be stabilized.

4.2 System evolution

In order to understand the evolution of the system it's useful to divide the system in two parts : **free response** and **forced response**, this can happen when the system is linear, because for definition a system is linear when the effect principle is valid.

At this point starting from a system like this(I write only the state, but this is also valid for the output) :

$$x(t) = \chi(t, t_0, x, x_0, u_{[t_0, t)}) \quad (4.8)$$

It's possible to obtain this one :

$$x(t) = \chi(t, t_0, x, x_0, 0) + \chi(t, t_0, x, 0_x, u_{[t_0, t)}) \quad (4.9)$$

The first part depends only from initial condition and is called **free response** because the system is free to evolve without other input, instead the second one is the **forced response** that doesn't depends on initial conditions, but only from input.

Let's transfer in another representation, in particular in the Lagrange form :

$$x(t) = e^{At}x_0 + \int_t^{t_0} e^{A(t-\tau)}Bu(\tau)d\tau \quad (4.10)$$

Free Response

The free response represents the natural evolution of the system from an initial state x_0 without external influence ($u(t) = 0$). Mathematically, it is described by the matrix exponential:

$$x_{free}(t) = e^{At}x_0 = \sum_{i=1}^n c_i e^{\lambda_i t} v_i \quad (4.11)$$

where λ_i and v_i are the eigenvalues and eigenvectors of the state matrix A . As previously calculated, the system possesses one eigenvalue in the right-half plane ($\lambda \approx 4.09$). This indicates that the free response is dominated by an unstable mode. Even for a very small initial tilt θ_0 , the state will grow exponentially along the direction of the associated eigenvector v_1 , causing the person to fall. The other two eigenvalues are stable, but the overall system remains unstable because at least one mode diverges.

To further analyze the instability, we examine the eigenvectors associated with the eigenvalues. Using the MATLAB command $[V, D] = \text{eig}(A)$, we obtain the modal matrix V . The first column v_1 , corresponding to $\lambda_1 = 4.0969$, represents the unstable mode shape:

$$V = \begin{bmatrix} 0.2371 & -0.2302 & -0.0005 \\ 0.9715 & 0.9731 & 0.0046 \\ 0 & 0 & -10 \end{bmatrix} \quad (4.12)$$

The first column shows that the unstable divergence is characterized by a specific ratio between the angle (x_1) and the angular velocity (x_2). In our specific project, this means that if the system is slightly perturbed, the tilt and the velocity will grow proportionally along this direction, leading to a catastrophic fall. Note that the third component is zero, meaning the muscle activation (x_3) does not naturally contribute to the unstable geometric divergence of the pendulum.

Forced Response

The forced response describes how the system reacts to an external input $u(t)$ starting from an equilibrium state ($x_0 = 0$). It is governed by the convolution integral:

$$x_{forced}(t) = \int_0^t e^{A(t-\tau)} B u(\tau) d\tau \quad (4.13)$$

For an unstable system, the forced response to common inputs like a unit step (constant torque) or an impulse (a sudden push) will also diverge. This is due to the fact that the input energy 'excites' the unstable internal mode of the

matrix A . In our model, the forced response illustrates that any external disturbance, if not countered by a specific control strategy, will move the 'human pendulum' away from the vertical position irreversibly.

It's possible to notice the instability also observing how system respond at some signal, for example with an **impulse signal**, in particular this type of signal is called **Dirac Impulse** and the response give to us so much information.

$$u(t) = \delta(t) = \lim_{\epsilon \rightarrow 0} \delta_{\epsilon}(t) \quad (4.14)$$

In matlab :

```
1 figure;
2 impulse(sys);
3 grid on;
```

Listing 4.2: MATLAB Script for State-Space Analysis

In this way is also possible to see the graph :

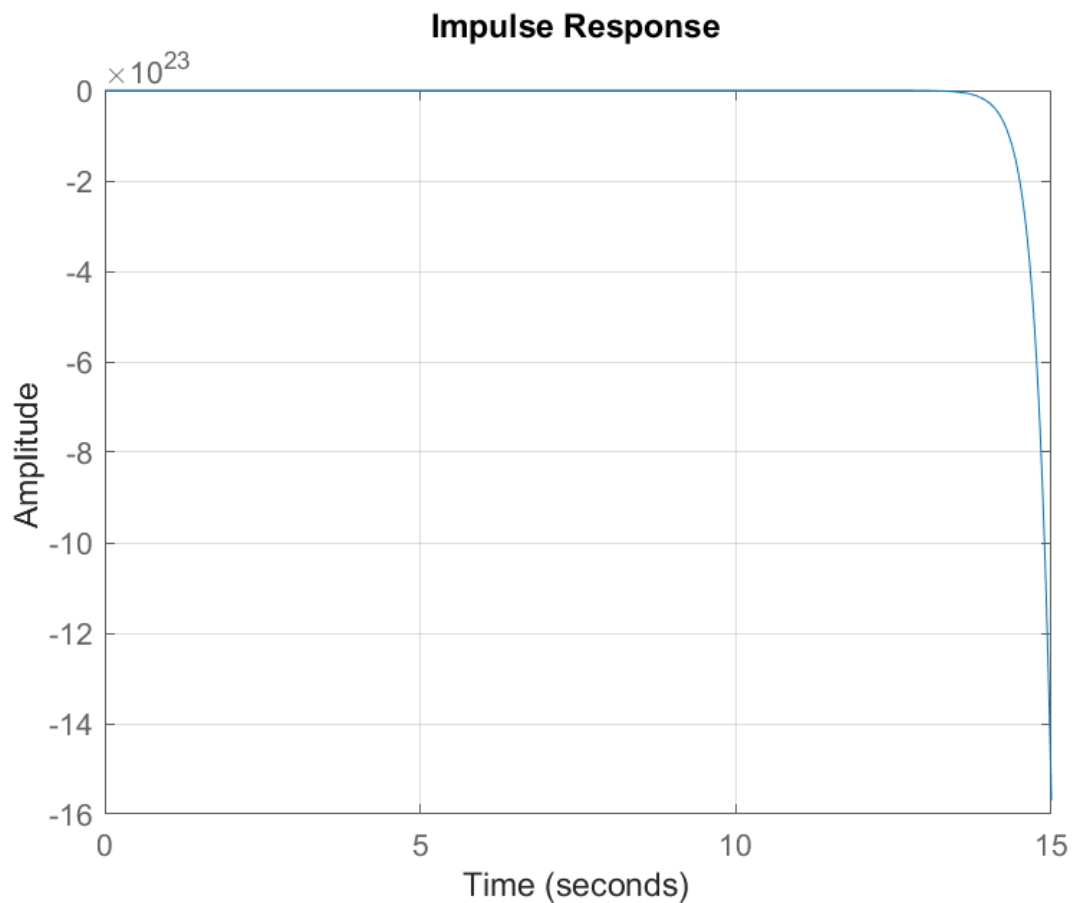


Figura 4.1: Impulsive response

It's possible to see that the response diverges.

Capitolo 5

Synthesis

Now that analysis is done, there is another thing that we have to do, and is the **synthesis** of a controller capable to, with a linear feedback of the state, allows us to bring the closed loop system to asymptotically stability.

But, at the first, to see if we can design a controller, we have to check some **structural properties** of the system, in particular property of **reachability** and property of **observability**.

In addition to the **feedback controller**, will be also designed an **asymptotic observer**.

5.1 Structural properties

5.1.1 Reachability

This property answer at the question : *"starting from a null initial state, which are all the state reachable?"*.

In the first is useful to write the state equation in Lagrange form, considering that we started from a null initial state so :

$$x(t) = \int_0^t e^{A(t-\tau)} Bu(\tau) d\tau \quad (5.1)$$

$x(t)$ is said **reachable** in the interval $[0, t]$ if exists an input, $u(\sigma)$ with $\sigma \in [0, t]$ such that the above equation is satisfied; and all the states that are reachable are in this set :

$$X_{[0,t]}^r \quad (5.2)$$

Now, in order to obtain the **reachability matrix** start from the process of the equation :

$$x = \int_0^t [\beta_0(t-\tau)I + \beta_1(t-\tau)A + \dots + \beta_n(t-\tau)A^{n-1}]Bu(\tau)d\tau \quad (5.3)$$

Then we obtain :

$$R = \begin{bmatrix} B & AB & \dots & A^{n-1}B \end{bmatrix} \quad (5.4)$$

Since this matrix doesn't depends on τ we can bring it outside the integrator so :

$$R \int_0^t \begin{bmatrix} \beta_0(t-\tau) \\ \dots \\ \beta_{n-1}(t-\tau) \end{bmatrix} u(\tau)d\tau = R \begin{bmatrix} \alpha_1(t) \\ \dots \\ \alpha_n(t) \end{bmatrix} \quad (5.5)$$

So we can conclude that x is reachable if satisfied this equation :

$$x = R \begin{bmatrix} \alpha_1(t) \\ \dots \\ \alpha_n(t) \end{bmatrix} \quad (5.6)$$

$$X^r = Im(R) \quad (5.7)$$

To have the **full reachability** the rank of R must be equal to "n", another important thing in continuos time is that the set of reachable states is equal to the set of controllability states.

A system is controllable if exists $u_0, u_1, \dots, u_{T-1}$ such that :

$$0 = A^T x + R \begin{bmatrix} u_{T-1} \\ u_{T-2} \\ \dots \\ u_0 \end{bmatrix}$$

$$\Rightarrow -A^T x = R \begin{bmatrix} u_{T-1} \\ u_{T-2} \\ \dots \\ u_0 \end{bmatrix} \quad (5.8)$$

Condition : $Im(R) = \mathbb{R}^{\kappa} \Rightarrow X_T^C = \mathbb{R}^{\kappa}$

But, it could be controllable even if is not fully reachable, because we don't need to generate every state, but the ones that are generated starting from the free response.

Matlab implementation

To check this in matlab this is the script :

```
1 %controllability
2 R = ctrb(A,B)
3
4 %computation of the rank
5 RankR = rank(R)
```

Listing 5.1: MATLAB Script for State-Space Analysis

As stated before controllability and reachability in continuous time are the same, this is the reason why the command is `ctrb()`.

Result is :

$$1 \times 10^3 * \begin{bmatrix} 0 & 0 & -0.0004 \\ 0 & -0.0004 & 0.0038 \\ 0.0100 & -0.1 & 1 \end{bmatrix}, \quad rank(R) = 3 \quad (5.9)$$

5.1.2 Observability

Observability answer to the question : "it's possible to compute the initial state from data(u and y) and model without ambiguity"?

Let's write the system in terms of free and forced response, remembering from the previous property that the forced response could be written also using the reachability matrix.

$$\begin{cases} x(t) = e^{At}x_0 + \int_0^t e^{A(t-\tau)}Bu(\tau)d\tau + Du(t) \\ y(t) = Ce^{At}x_0 + \int_0^t Ce^{A(t-\tau)}Bu(\tau)d\tau + Du(t) \end{cases} \quad (5.10)$$

An initial state x is not observable if it is indistinguishable in the future in t -step by 0_x

Indistinguishable means :

$$x - 0_x \in \ker \begin{bmatrix} C \\ CA \\ \dots \\ CA^{n-1} \end{bmatrix} \quad (5.11)$$

So x is indistinguishable in the future by 0_x if $x \in \ker(O)$, where O it's called **observability matrix**, all the states that respect this condition are in the set of the states **not observable**, and in order to reach the full observability we want this set empty so the condition for the full observability is :

$$\text{rank}(O) = n \quad (5.12)$$

Matlab implementation

```
1 %observability matrix
2 O = obsv(A,C)
3
4 %Rank of O in order to chek the full observability
5 RankO = rank(O)
```

Listing 5.2: MATLAB Script for State-Space Analysis

Results are :

$$O = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 17.3211 & -0.1310 & -0.0374 \end{bmatrix} \quad (5.13)$$

and

RankO = 3.

5.2 Static State feedback

The open-loop analysis of the system revealed an inherent instability, characterized by an eigenvalue located in the right-half of the complex plane ($\lambda \approx 4.09$).

To stabilize the "human pendulum" and ensure it maintains an upright equilibrium position, a feedback control law must be implemented. For this project, a Static State Feedback approach has been chosen. In this configuration, the control input $u(t)$ is calculated as a linear combination of all the system's state variables:

$$u(t) = kx(t) \quad (5.14)$$

with $k \in \mathbb{R}^n$.

Having the system written in state space representation :

$$\begin{cases} \dot{x} = Ax + Bu \\ y = Cx + Du \end{cases} \quad (5.15)$$

If we substitute, and if we consider that this system has $D = 0$, it becomes :

$$\begin{cases} \dot{x} = (A + Bk)x \\ y = Cx \end{cases} \quad (5.16)$$

It's possible to observe that the closed loop system has a dynamics depending on the new state matrix.

Since the system, as we have seen, is fully reachable, it's possible to allocate the eigenvalues in order to make system asymptotically stable.

The eigenvalues that have been chosen for the matrix are:

$$\lambda_1 = -2.5, \quad \lambda_2 = -3, \quad \lambda_3 = -10 \quad (5.17)$$

To design k , it was used the Ackerman formula, with this result :

$$k = \begin{bmatrix} 911.6308 & 207.8847 & -0.5369 \end{bmatrix} \quad (5.18)$$

5.2.1 Matlab and simulink

For the simulink implementation it's used the model of the linearized system, defined previously, and then add the control law, in order to do this, on the matlab script is applied the ackerman formula and then, checked the eigenvalues, but also full reachability and observability, because, the reachability is guaranteed by the reachability of the open loop system, but we can't say the same for the observability because there may be some cancellation.

```
1
2 %% CONTROLLER
3
4 K_ack = -acker(A,B, [-2.5 -3 -10])
5 eigen_ack = eig(A + B*K_ack)
6 sim("Dynamical System theory\Progettino\Matlab\ControllerFSF.
   slx")
7
8 %if the system in open loop is reachable, the closed loop will
   also be, but
9 %the observability could be lost, so check also the obs
10 R_cl = ctrb(A +B*K_ack,B)
11 O_cl = obsv(A +B*K_ack,C)
12 rank_r_cl = rank(R_cl)
13 rank_o_cl = rank(O_cl)
```

Listing 5.3: MATLAB Script for State-Space Analysis

Results :

```
eig_2_1 =
-10.0000
-2.5000
-3.0000

R_cl =
1.0e+03 *
    0         0 -0.0004
    0 -0.0004  0.0058
    0.0100 -0.1537  1.5840

O_cl =
    1.0000         0         0
         0    1.0000         0
    17.3211 -0.1310 -0.0374
```

Figura 5.1: Results of control system

In simulink blocks are defined in this way :

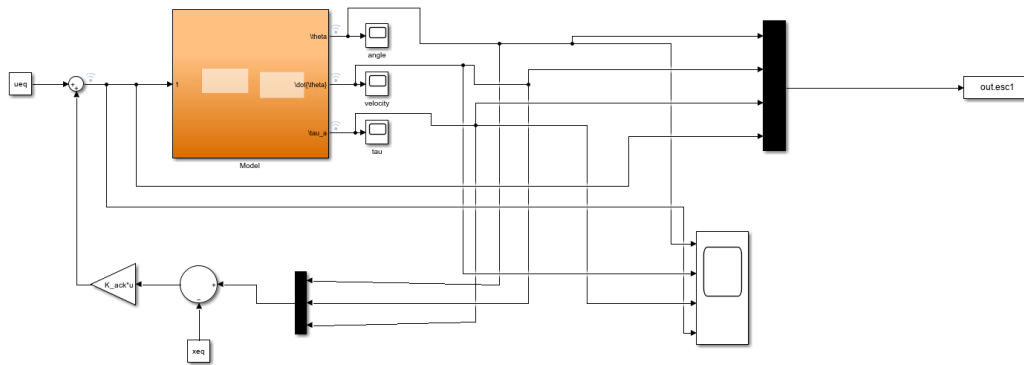


Figura 5.2: Simulink controller implementation

In order to show how the closed loop change the behave of the system, let's observe the impulse response, and compare it with the open loop one :

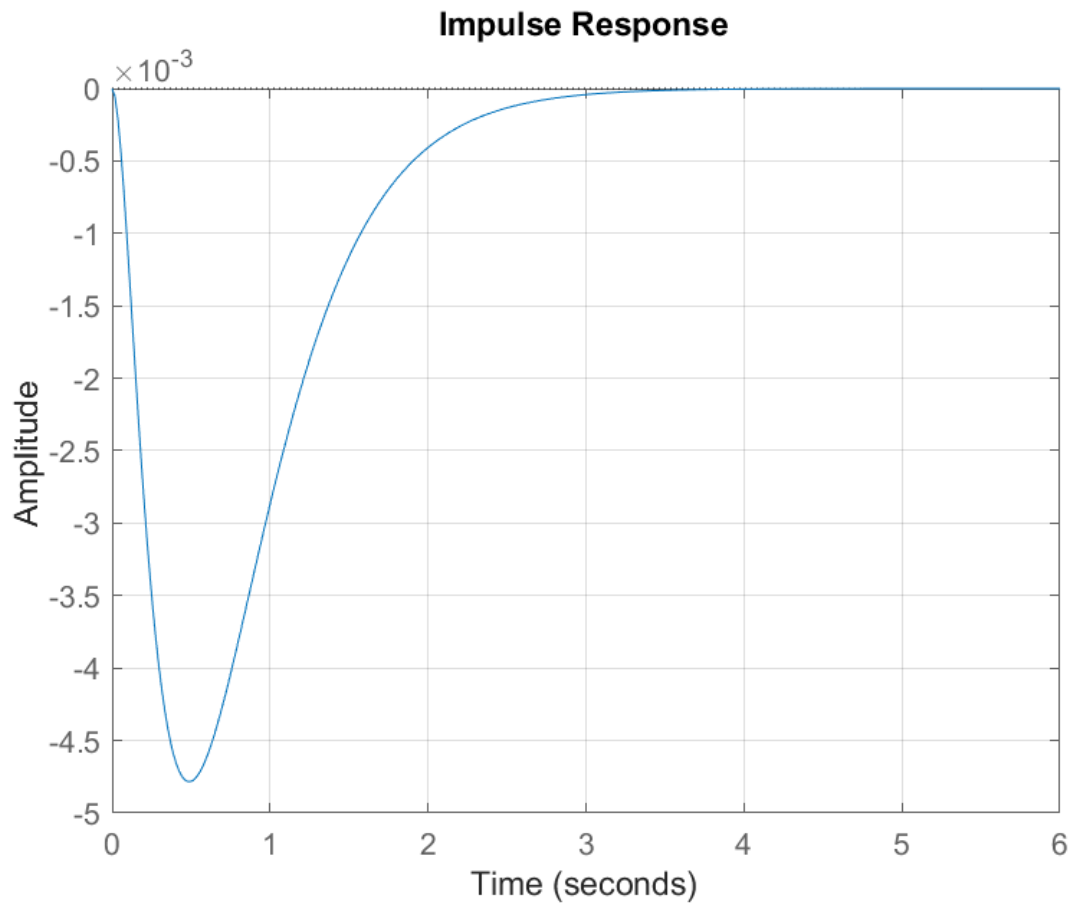


Figura 5.3: Impulse response of closed loop system

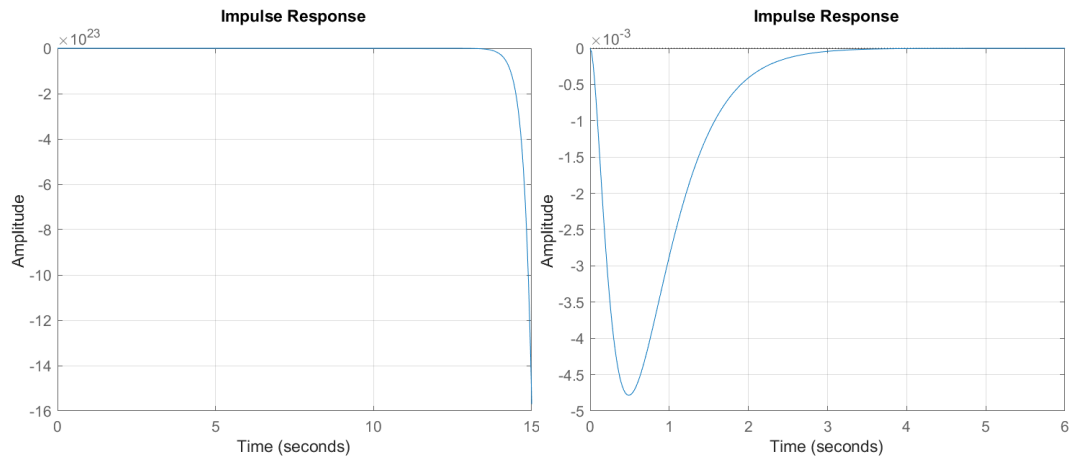


Figura 5.4: Comparison

The response of the closed loop converges to 0, this means that the controller works, instead the response of the system without controller diverges, it's like someone gives a light push to a person and they fall, but with the controller the posture remained upright.

To evaluate the performance of the controller, a non-zero initial condition ($\theta_0 = 0.1$ rad) was imposed on the plant's dynamics, and those are the plots:

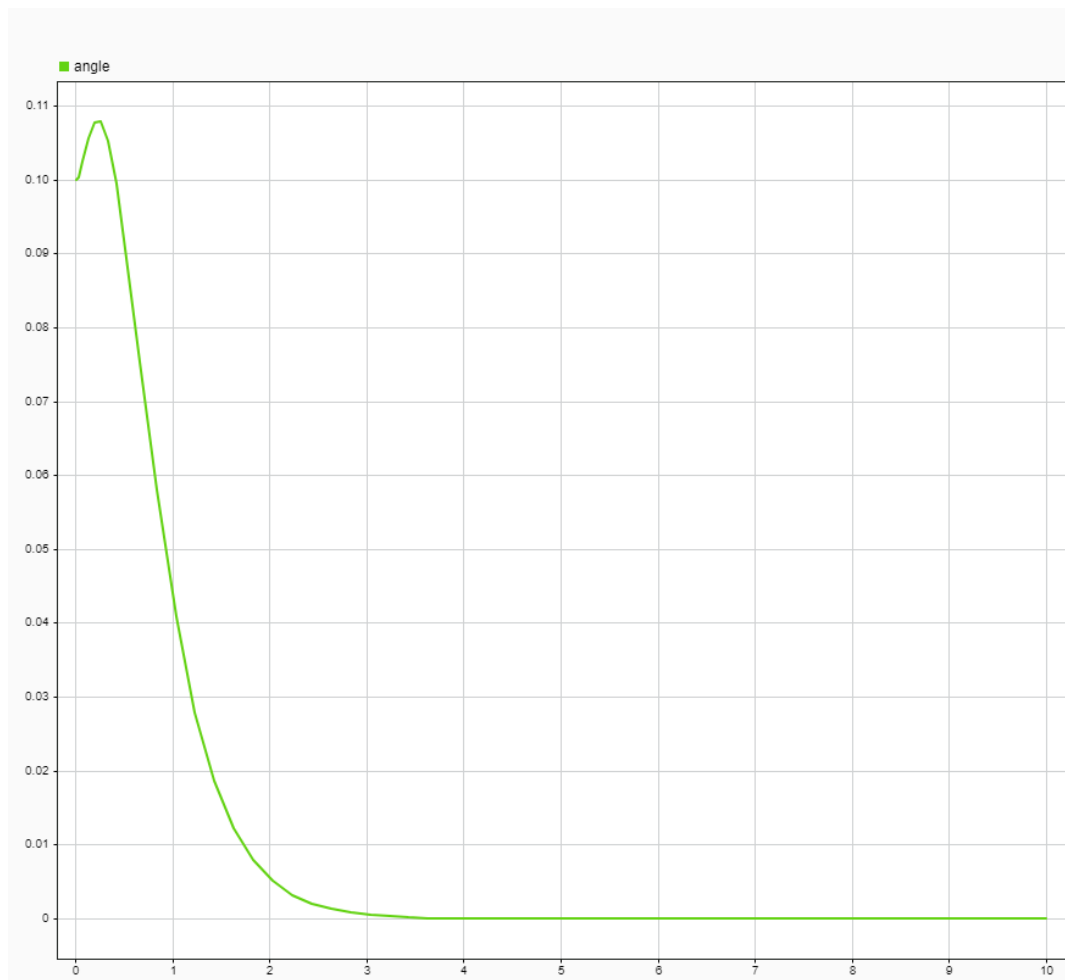


Figura 5.5: Angle with controller

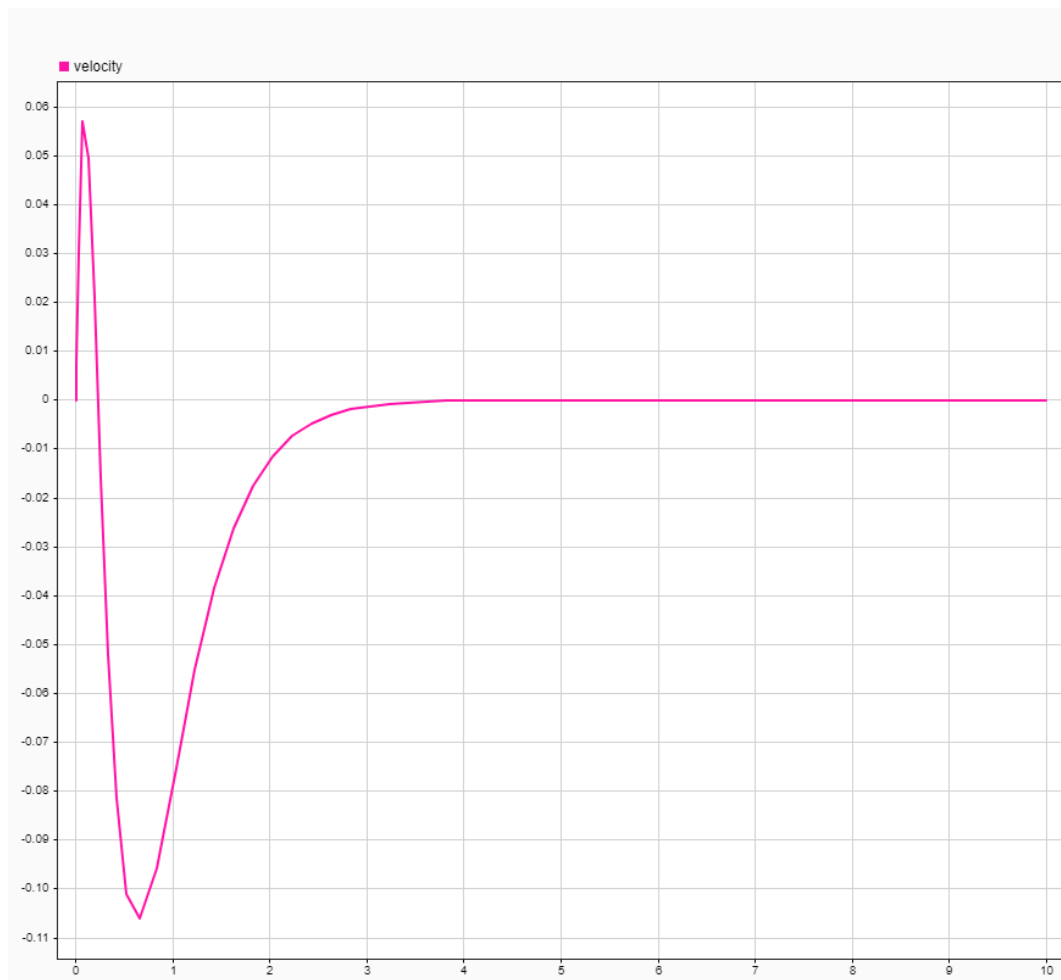


Figura 5.6: Velocity with controller

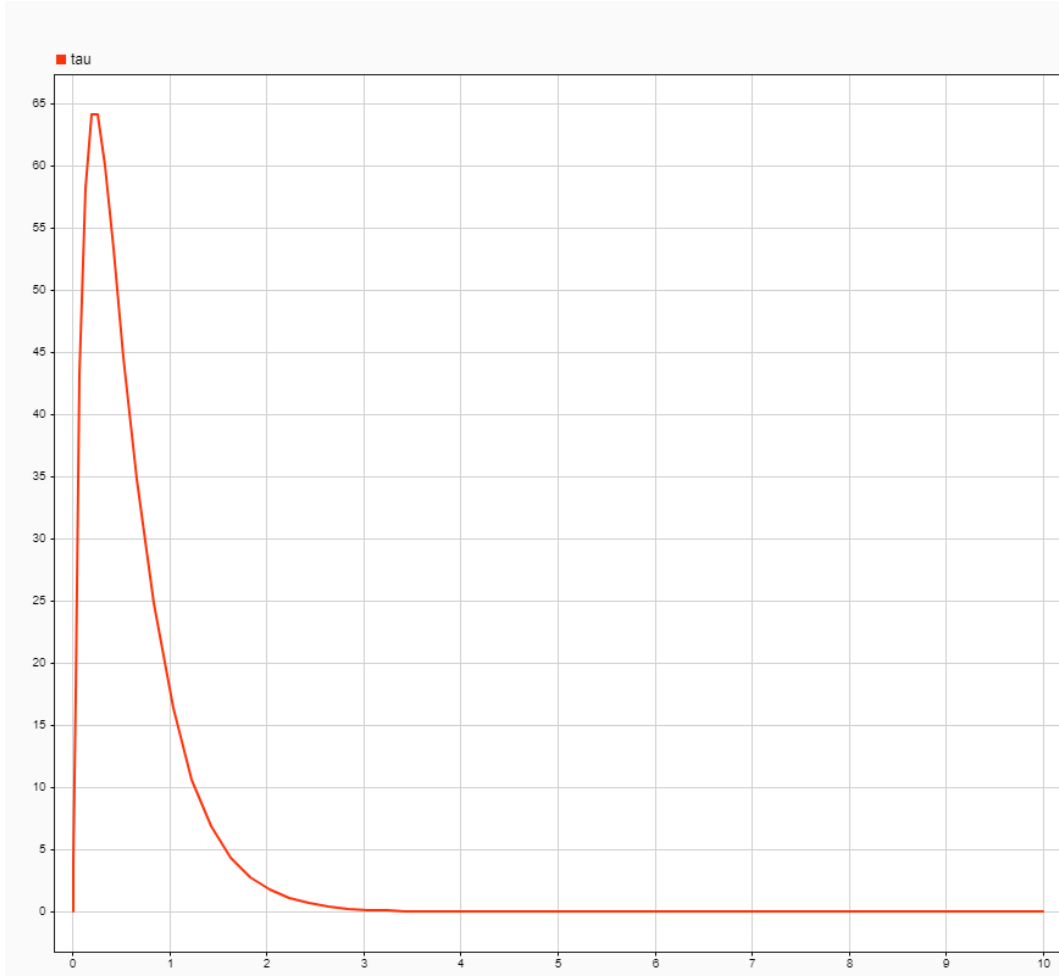


Figura 5.7: Muscle activation with controller

5.3 Asymptotic observer(Luenberger)

Asymptotic observer is a particular dynamical system that helps when is not possible to measure all the states of the system, this is a condition that occurs very often, because, measuring all the states implies having a lot of sensors, and sensors are not very convenient in terms of costs, but also in terms of disturbance that they applies to the system. For those reasons the asymptotic observer is built like an estimator of the state, and as such, it will not measure exactly so there will be some error.

In our specific case, while the sensor only provides the angular position θ , the observer allows the controller to access the angular velocity and muscle activation states, which are essential for stabilizing the human pendulum.

Starting from the system(plant) in time continuous :

$$\begin{cases} \dot{x}(t) = Ax(t) + Bu(t) \\ y(t) = Cx(t) + Du(t) \end{cases} \quad (5.19)$$

And then the observer one :

$$\begin{cases} \dot{\hat{x}}(t) = A\hat{x}(t) + Bu(t) + L(\hat{y}(t) - y(t)) \\ \hat{y}(t) = C\hat{x}(t) + Du(t) \end{cases} \quad (5.20)$$

The thing that we want is that :

$$\lim_{\epsilon \rightarrow \infty} e(t) = O_n \quad (5.21)$$

If we do some mathematical manipulation starting from the evolution of the error, we arrive at the conclusion that the error goes to zero when matrix $(A+LC)$ is asymptotically stable; where L is the **Observer gain** that multiply the output error in order to produce the correction action. In the practice, if we are in simulink, we have to take the controller scheme, but now consider also the estimation and change the poles of observer, so :

```

1 %design the observer
2 %define the poles
3 polesObs = [-15 -20 -30];
4
5 L = acker(A', C', polesObs);
6 L = L';
7
8 eig0 = eig(A-L*C);
9
10 Ao = A - L*C;
11 Bo = [B L];
12 Co = eye(size(A));
13 Do = zeros(size(A,1),2);
14
15
16 modelSim = sim('ControllerObserver.slx');
```

Listing 5.4: MATLAB Script for State-Space Analysis

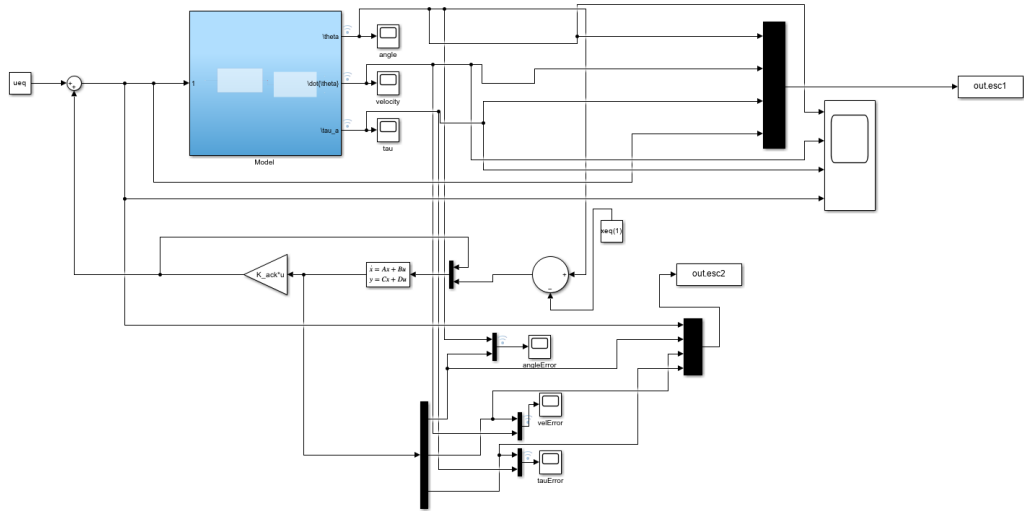


Figure 5.8: Scheme of Asymptotic Observer

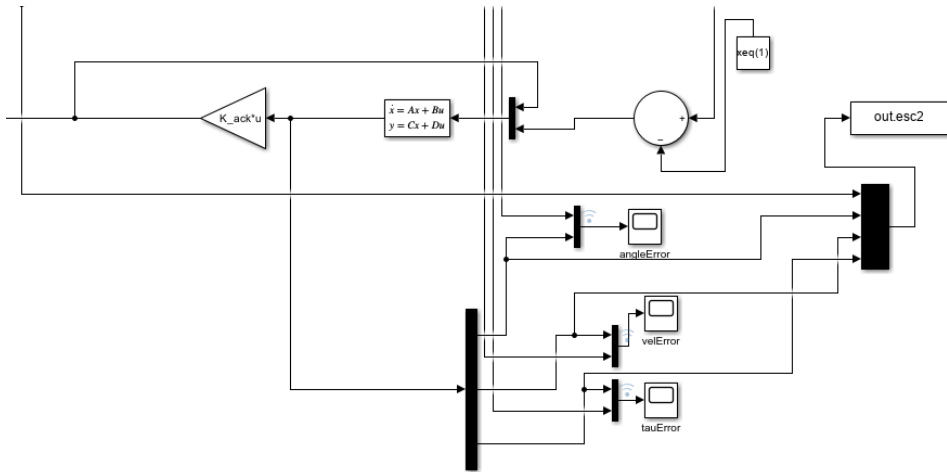


Figure 5.9: Scheme of Asymptotic Observer zoom on the error part

Those are the plots, that compares the estimated(blue line) and the real(yellow line) state :

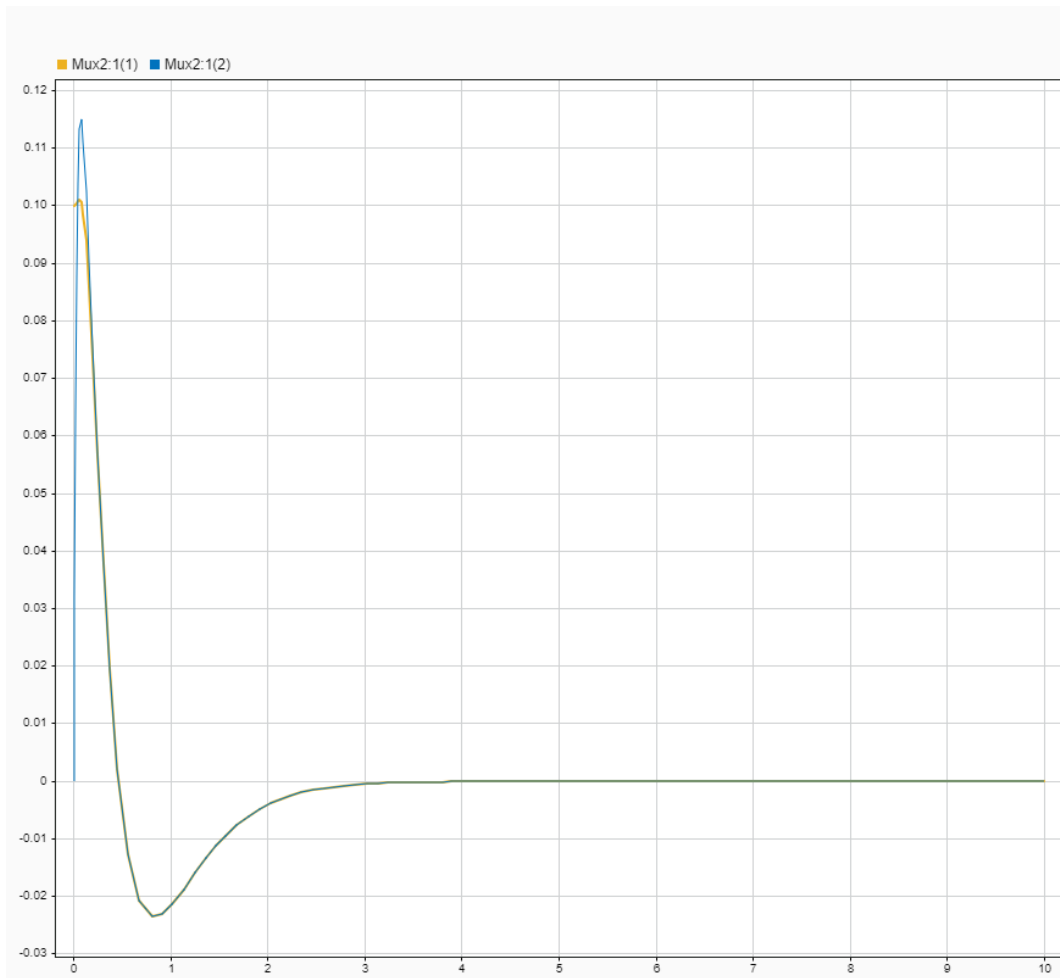


Figura 5.10: Comparison between real and estimated angle

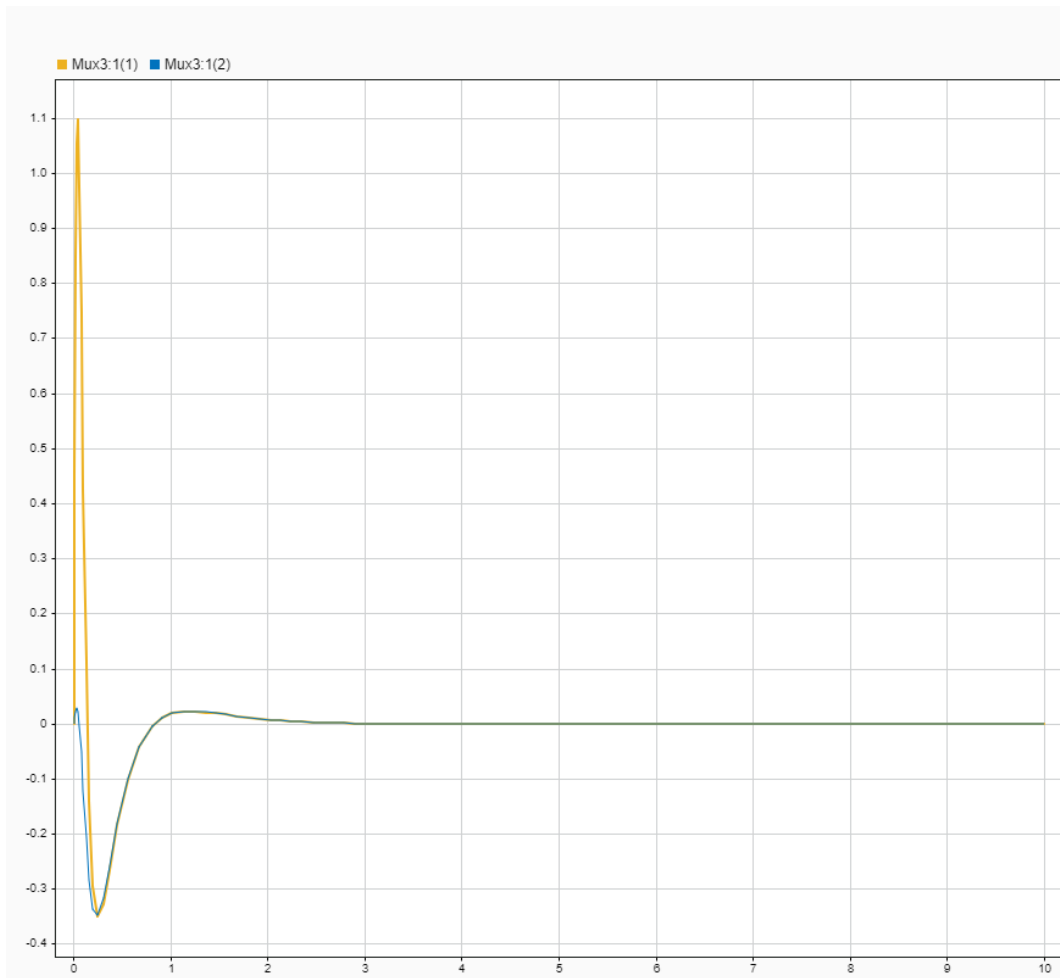


Figura 5.11: Comparison between real and estimated angular velocity

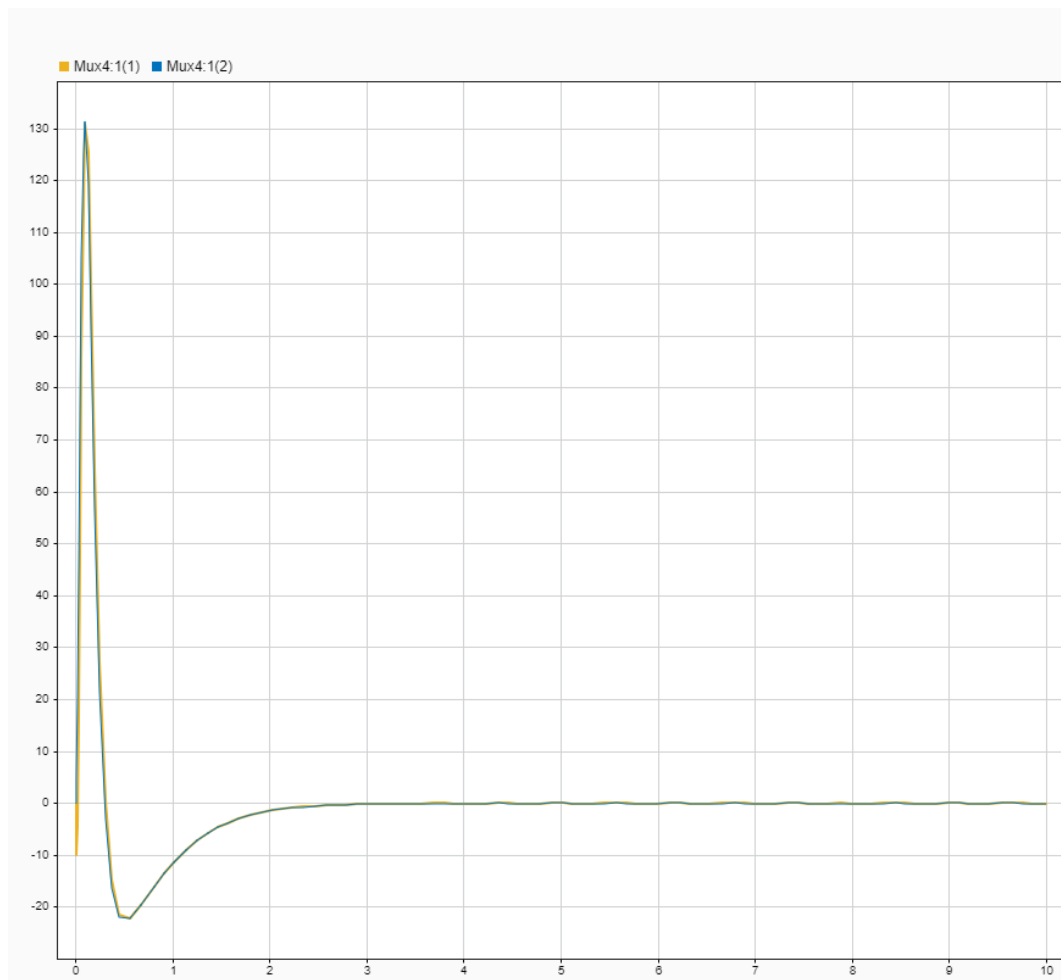


Figura 5.12: Comparison between real and estimated muscular activation

Capitolo 6

Conclusion

In this project, the problem of human postural control during quiet standing has been investigated through the lens of dynamical systems and control theory. By modeling the human body as an inverted pendulum actuated at the ankle joint, the inherently unstable nature of the upright posture has been highlighted.

Starting from a non-linear biomechanical model, the system dynamics were derived and implemented in MATLAB and Simulink. The equilibrium analysis revealed the presence of both stable and unstable configurations, with particular attention devoted to the upright equilibrium corresponding to human standing. Linearization around this operating point enabled the application of classical linear system analysis tools.

The open-loop analysis confirmed the instability of the upright posture, motivating the design of a stabilizing feedback controller. A static state feedback control law was successfully synthesized, ensuring asymptotic stability of the closed-loop system. Simulation results demonstrated the controller's effectiveness in rejecting perturbations and maintaining balance.

Furthermore, an asymptotic state observer was designed to estimate unmeasured states, reflecting realistic sensing limitations in human motor control. The observer exhibited fast convergence and accurate state reconstruction, validating the overall control architecture.

Despite the simplifying assumptions of the inverted pendulum model, the results illustrate how feedback mechanisms are fundamental to postural stability. This study provides a clear example of how control theory can be applied

to biomechanical systems, offering insight into both engineering design and human motor behavior.

Bibliografia

- [1] WINTER, David A. Biomechanics and motor control of human movement. John wiley & sons, 2009.
- [2] Prof Casavola, A. (s.d.). Appunti del Corso di Teoria dei Sistemi.
- [3] Peterka, R. J. (2000). Postural control model interpretation of stabilogram diffusion analysis. Biological Cybernetics, 82(4), 335-343.
- [4] KOT, Andrzej; NAWROCKA, Agata. Modeling of human balance as an inverted pendulum. In: Proceedings of the 2014 15th International Carpathian Control Conference (ICCC). IEEE, 2014. p. 254-257.